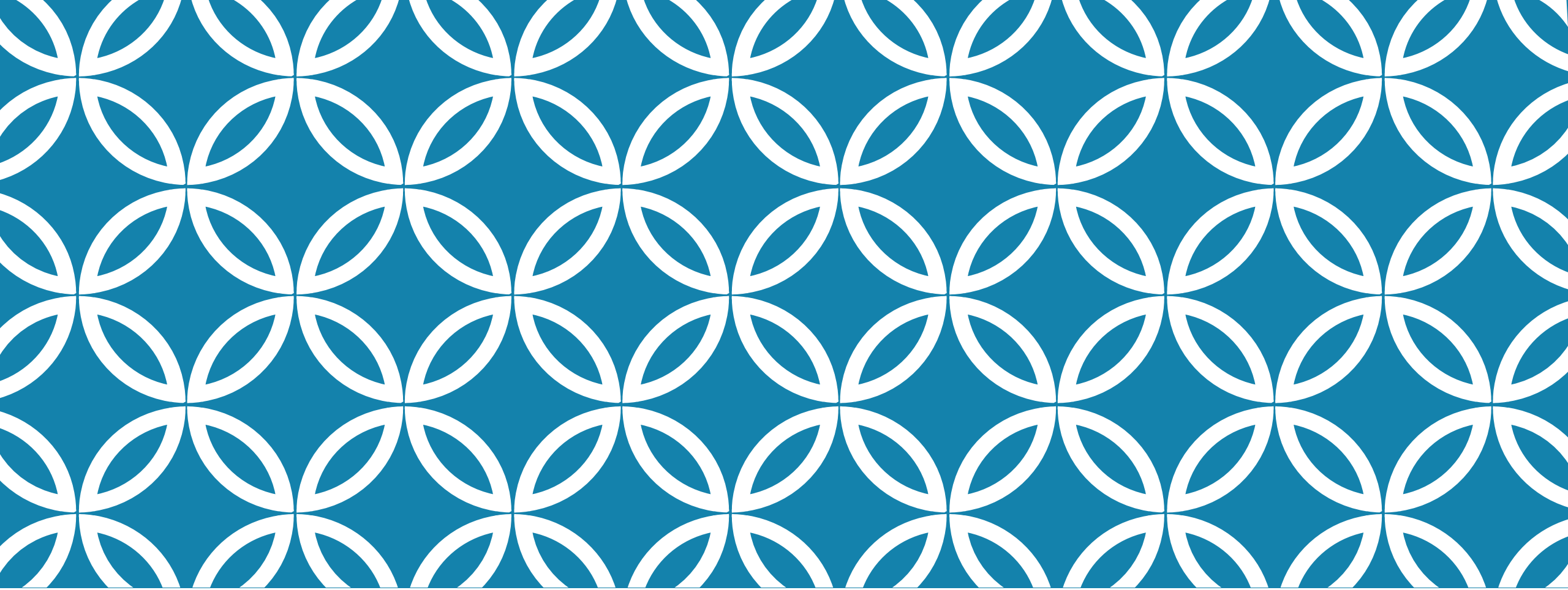


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

DİZİLER (ARRAY)

- Yazılım geliştiriciler, kodlarında farklı değişkenler tanımlama ihtiyacı duyar.
- Genellikle değişkenlere tek bir değer atanırken, aynı türdeki değişkenleri bir arada tutmak için diziler (array) kullanılır.
- Diziler, ortak adla anılan aynı tipte verilerin topluluğudur ve programlamada önemli bir veri yapısıdır.

DİZİLER

- Diziler, aynı veri tipindeki değişkenleri tek bir isim altında saklamamızı sağlar.
- **"Array"** terimi, ortak bir isimle anılan aynı türdeki verilerin topluluğunu ifade eder.
- Örneğin, 8 adet meyve adını bir listede tutmak için bir dizi kullanılabilir.

DİZİ OLUŞTURMA

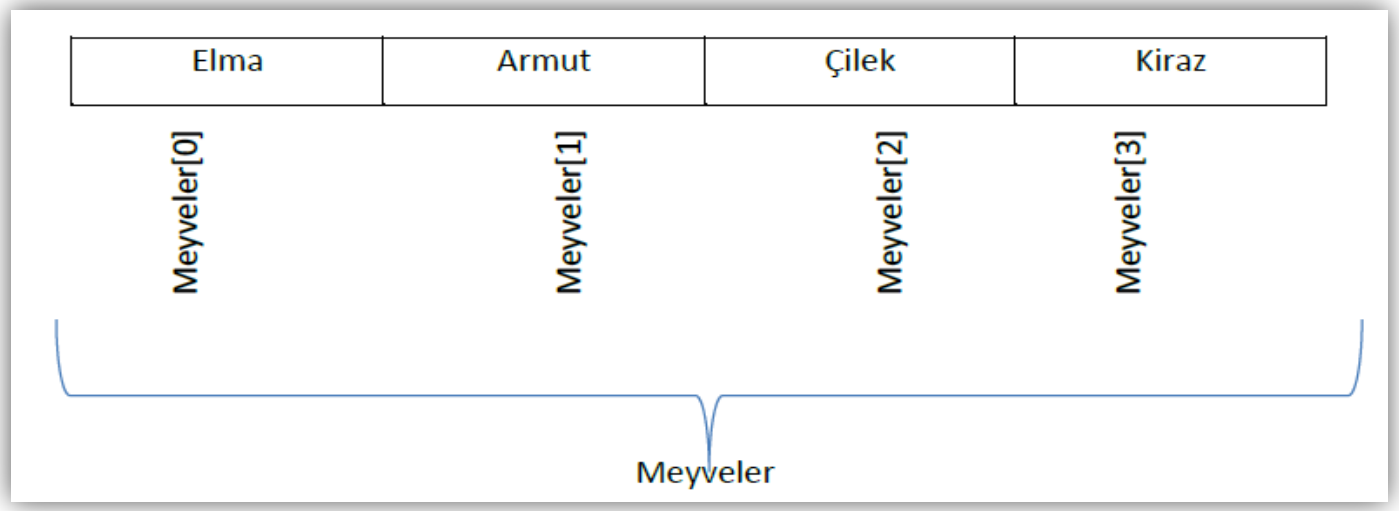
- Dizi oluştururken, köşeli parantezler ve bir değişken ismi ile elemanların veri tipleri belirlenir.
- İki yöntemle dizi tanımlanabilir:
- **1. Yöntem:**
  - ***degiskenturu[ ] dizi_ismi = new degiskenturu[elemansayisi]***
- Burada, dizinin veri türü belirlenir ve köşeli parantezler bir boyutlu dizinin tanımlandığını gösterir.
- ***"elemansayisi"*** ile dizi içinde ne kadar eleman olacağı belirtilir.

DİZİ OLUŞTURMA

- Örnek bir dizi tanımlama aşağıdaki gibidir:
- ***string[ ] Meyveler = new string[4];***
- Yukarıda verilen dizi örneğinde meyveler isimli dizi içerisine **string** türünde **4 adet** meyve verisi tutabiliriz.

DİZİ OLUŞTURMA

- Oluşturulan dizi kavramsal olarak aşağıdaki gibi gösterilir.



DİZİ OLUŞTURMA

- **2. Yöntem:**
- Eğer **int** türünde eleman içeren bir dizi tanımlamak isteniyorsa, aşağıdaki gibi bir yapı kullanılabilir.
- Burada "**finalnotları**" adlı dizi, **5** adet **int** türünde veri tutacak şekilde oluşturulmuştur.

```
int[] finalnotları;
finalnotları = new int[5];
```

DİZİYE DEĞER GİRME

- Dizi tanımlandıktan sonra, elemanlara değer atamak için ***indeks*** değerleri kullanılır.
- Dizi indeksi, bir elemanın dizideki konumunu belirtir.
- Genellikle dizinin ilk indeksi **0**’dır.
- Örneğin, 7 elemanlı bir dizide indeksler 0'dan 6'ya kadar uzanır.
- Dizi tanımlama esnasında veya program akışında değerler atanabilir.

DİZİYE DEĞER GİRME

- **Örnek 1: int Dizisi**

```
0 başvuru
static void Main(string[] args)
{
    int[] final_notlari = new int[3];
    final_notlari[0] = 65;
    final_notlari[1] = 85;
    final_notlari[2] = 75;
}
```

DİZİYE DEĞER GİRME

- **Örnek 2: char Dizisi**

```
char[] harfler = new char[6];
harfler[0] = 'A';
harfler[1] = 'T';
harfler[2] = 'A';
harfler[3] = 'A';
harfler[4] = 'Ö';
harfler[5] = 'F';
```

DİZİYE DEĞER GİRME

- Dizi sınırlarına dikkat etmek önemlidir; eleman sayısından fazla değer girilmeye çalışılırsa program hata verecektir.
- **Çift Sayı Dizisi Örneği:**

```
int[] ciftSayilarimiz = new int[5];
int i = 0;
for (int j = 0; j <= 21; j += 2)
{
    ciftSayilarimiz[i] = j;
    i++;
}
```

DİZİYİ YAZDIRMA

- Dizi elemanlarına erişmek için indeks numarası kullanılır.
- Örneğin, diziyi ekrana yazdırmak için:

```
string[] iller = new string[10];
iller[0] = "Adana";
iller[4] = "Ankara";
iller[6] = "İstanbul";
iller[8] = "Erzurum";

Console.WriteLine(iller[4]);
Console.WriteLine(iller[6]);
Console.WriteLine(iller[8]);
```

Output

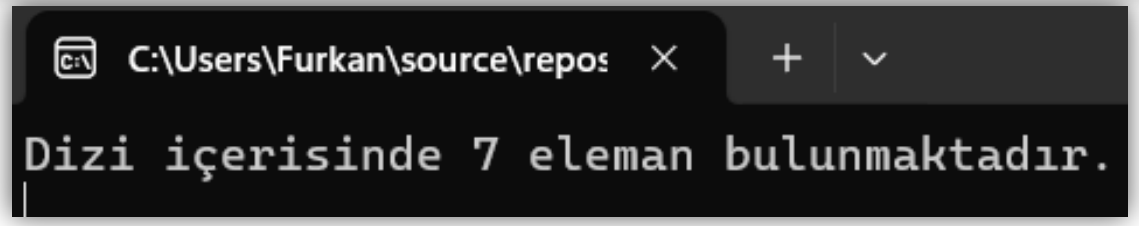
```
Ankara
İstanbul
Erzurum
```

DİZİ ÖZELLİKLERİ VE METOTLARI

- **Length:** Bir dizinin toplam eleman sayısını verir.

```
0 başvuru
static void Main(string[] args)
{
    int[] tek_sayilar = new int[7];
    int eleman_sayisi = tek_sayilar.Length;
    Console.WriteLine("Dizi içerisinde {0} eleman bulunmaktadır.", eleman_sayisi);

    Console.ReadKey();
}
```



DİZİ ÖZELLİKLERİ VE METOTLARI

- **Clear:** Belirlenen indeksler arasındaki değerleri temizler.

```
int[] sayilar = new int[3];
sayilar[0] = 12;
sayilar[1] = 18;
sayilar[2] = 21;
Array.Clear(sayilar, 1, 2);
```

DİZİ ÖZELLİKLERİ VE METOTLARI

- **Reverse:** Dizi elemanlarının sırasını tersine çevirir.

```
0 başvuru
static void Main(string[] args)
{
    char[] kisa_alfabe = new char[6];
    int j = 0;
    for (char harfler = 'A'; harfler <= 'F'; harfler++)
    {
        kisa_alfabe[j] = harfler;
        j++;
    }
    Array.Reverse(kisa_alfabe);

    Console.WriteLine(kisa_alfabe);

    Console.ReadKey();
}
```